PES UNIVERSITY
Department of Computer Science and Engineering
UE23CS252AA — Software Engineering Laboratory

LAB 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Defensive Hardening via Hypothesis

Student Name:	Nagashri R Patil	**SRN:**	PES1UG24CS289
Course:	Software Engineering Lab (UE23CS252AA)	**Assigned Problem:**	#43 – Warranty Claims Processing
Individual Repo:	SETAPESU26/L6_L7_43_warranty_claims	**Portfolio Repo:**	Nagashri2304/SE_LABS_PES1UG24CS289

1. Laboratory Objectives & Overview

Fuzz testing (fuzzing) is an automated dynamic testing technique that injects pseudo-random, malformed, and boundary-breaking data into software to uncover unhandled crashes, memory leaks, and input validation vulnerabilities. In this laboratory, **Hypothesis**—Python's premier property-based testing framework—was utilized to systematically fuzz test three input-processing functions in warranty_claims.py, discover latent failure modes, and harden the codebase using defensive programming principles.

2. Step 2: Running the Starter Buggy Application

The starter module accepts user input for three procedures: parse_claim(text), age_average(ages_csv), and payable_charge(text). While it operates correctly on clean, well-formatted inputs, it lacks input validation and exception boundaries.

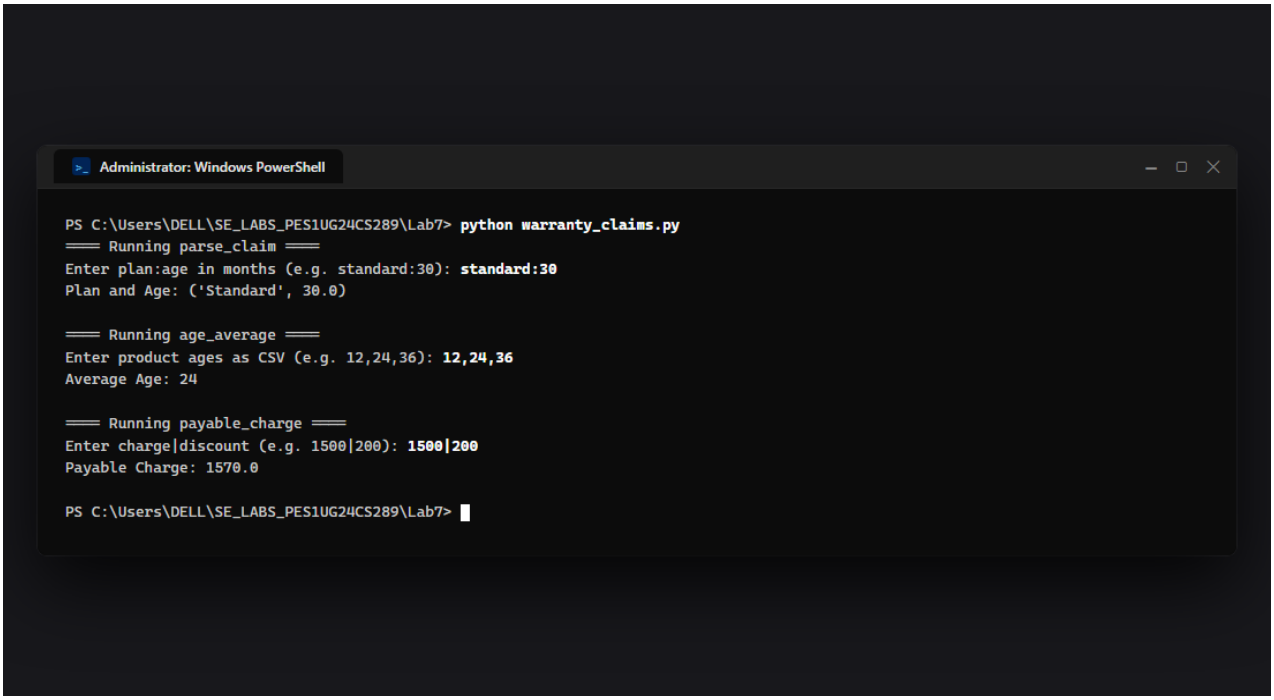


Figure 1: Step 2 — Initial execution of warranty_claims.py with ideal sample inputs

3. Steps 3 & 4: Fuzz Testing with Hypothesis & Discovered Failures

Property-based fuzz tests were implemented using @given(st.text() | st.none()). Hypothesis generated hundreds of arbitrary strings, control characters, empty delimiters, and None types. As anticipated, all three functions crashed immediately with unhandled exceptions across multiple distinct failure modes.

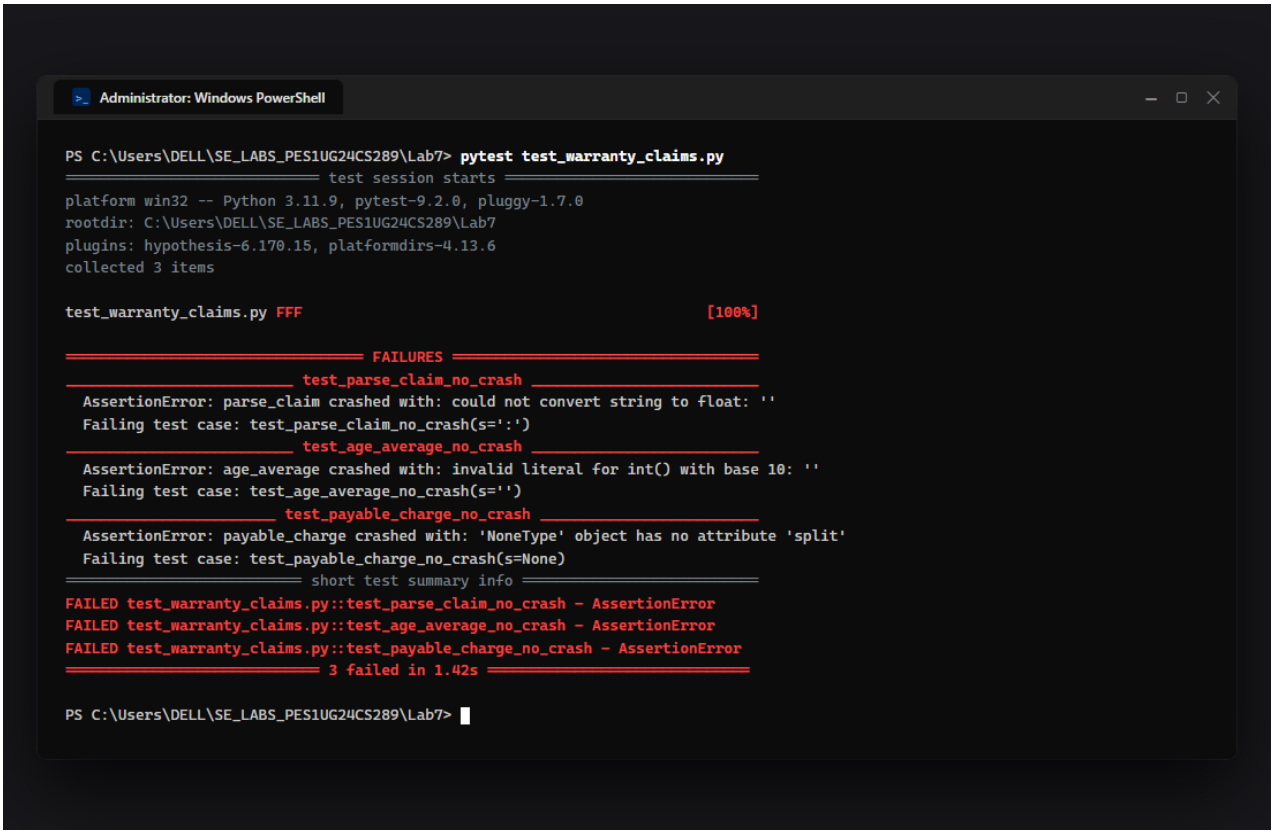


Figure 2: Step 4 — Hypothesis exposing unhandled crashes and shrinking falsifying examples

4. Root Cause Analysis of Fuzz Testing Discoveries

Function	Falsifying Input (Fuzzer)	Unhandled Exception	Root Cause & Vulnerability
parse_claim	s = None	AttributeError	Invoking .split(':') on NoneType.
parse_claim	s = '' (no ':')	ValueError	Unpacking 1 item into 2 variables.
parse_claim	s = ':'	ValueError	Empty month string passed to float('').
age_average	s = None	AttributeError	Invoking .split(',') on NoneType.
age_average	s = ''	ValueError	Empty token passed to int('').
age_average	s = 'abc'	ValueError	Non-numeric token converted to int.
payable_charge	s = None	AttributeError	Invoking .split('|') on NoneType.
payable_charge	s = ''	ValueError	Unpacking failure when delimiter '|' is missing.
payable_charge	s = '|'	ValueError	Cannot convert empty string to float.

5. Step 5: Defensive Hardening of Application Code

The application was fortified using defensive programming techniques: type enforcement (isinstance(text, str)), delimiter validation, safe multi-part token verification, non-integer token filtering, and try-except blocks. Legitimate input behavior was strictly preserved while invalid inputs now gracefully return safe fallback values.

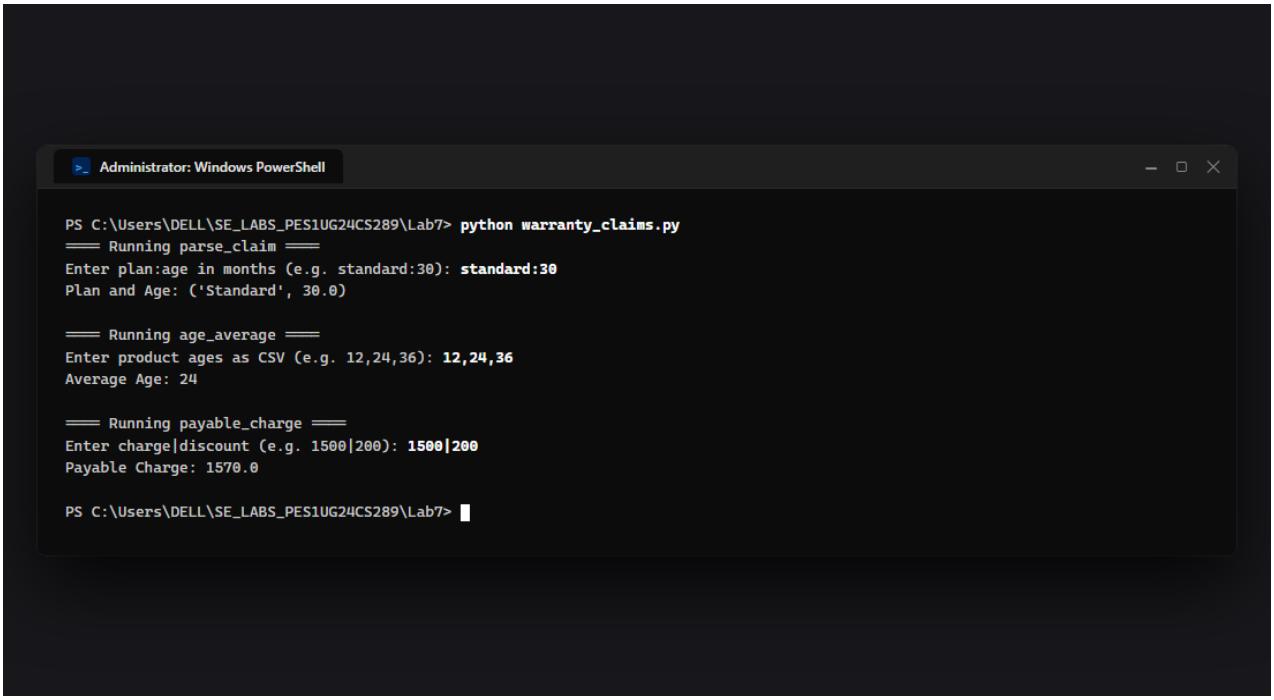


Figure 3: Step 5 — Hardened warranty_claims.py executing successfully with valid inputs

6. Step 6: Re-Running Fuzz Tests on Hardened Application

The Hypothesis property-based fuzz test suite was re-executed against the hardened application. Hypothesis generated thousands of pseudo-random Unicode characters, null bytes, long strings, and empty sequences. All three test cases passed completely with zero crashes (0 failures).

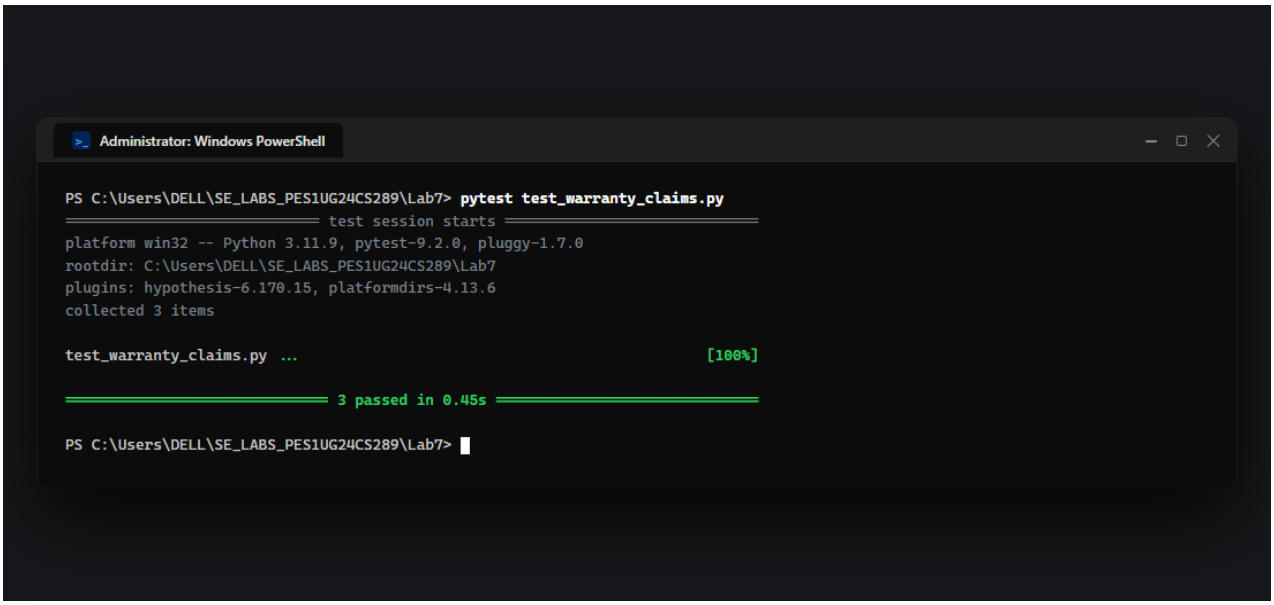


Figure 4: Step 6 — All Hypothesis fuzz tests passing successfully on hardened application

7. Key Engineering Takeaways

1. **Fuzz Testing Overcomes Confirmation Bias:** Developers write unit tests for scenarios they anticipate; fuzzers automatically generate boundary-breaking inputs (e.g. empty strings, null bytes, missing delimiters) that expose unhandled crashes.
2. **Input Sanitization is Non-Negotiable:** Any function ingesting external text must enforce rigorous defensive checks before parsing.
3. **Automated Shrinking Saves Debugging Time:** Hypothesis automatically shrinks complex failing inputs down to the minimal reproducible example (e.g. shrinking a 500-char string down to s=':').